

ゼロから python で行列の簡約化

MMA 平部員 B1 Tenpura

2026 年 9 月 20 日

1 はじめに

こんにちは。B1 平部員の Tenpura です。この記事では夏休みに重い腰を上げて知識ゼロから Python で行列式を書いてみた話をします。動機は Anki のアドオン作りで Python 書きの行き詰りによる萎え落ちを減らすことです。あと線形代数で追試くらったことです（単位はくれました）。雪辱を果たしに。

2 行列とは

行列は縦横に数を並べたものです。例えば
$$\begin{cases} x + 2y - z = 2 \\ 2x - y + 3z = 9 \\ 3x + y + z = 8 \end{cases} \quad \text{を} \quad \begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix}$$

のように書けます。シンプルですね。

3 簡約化とは

簡約化は、連立方程式を解く際に行列の値を都合よくする操作です。実際中学からの連立方程式の解き方まんまで、式同士の足し算、引き算、掛け算をします。例えば、3つの式を上から①②③として、

$$\begin{aligned} & \begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix} \xrightarrow[\text{③}-3\times\text{①}]{\text{②}-2\times\text{①}} \begin{bmatrix} 1 & 2 & -1 & 2 \\ 0 & -5 & 5 & 5 \\ 0 & -5 & 4 & 2 \end{bmatrix} \xrightarrow{\text{②}\times(-\frac{1}{5})} \begin{bmatrix} 1 & 2 & -1 & 2 \\ 0 & 1 & -1 & -1 \\ 0 & -5 & 4 & 2 \end{bmatrix} \\ & \xrightarrow[\text{③}+5\times\text{②}]{\text{①}-2\times\text{②}} \begin{bmatrix} 1 & 0 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & -1 & -3 \end{bmatrix} \xrightarrow{\text{③}\times(-1)} \begin{bmatrix} 1 & 0 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & 1 & 3 \end{bmatrix} \xrightarrow[\text{②}+\text{③}]{\text{①}-\text{③}} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} \end{aligned}$$

と変形します（行基本変形と言います）。最後の行列は

$$\begin{cases} x + 0 + 0 = 1 \\ 0 + y + 0 = 2 \\ 0 + 0 + z = 3 \end{cases}$$

となり、良い感じに文字の値が分かります。この操作を言語化すると、

1. すべての成分が 0 でない列のうち、一番左の列に注目する。
2. 注目した列の一番上の数が 0 のときは、0 でない行と入れ替える。
3. 第 1 行を何倍かして主成分をつくる。
4. 第 1 行を何倍かして他の行に加えて、主成分の上下にある成分をすべて 0 にする。
5. 第 1 行を忘れ、残った行で 1~4 を繰り返す。

となります。¹⁾我々 B1 は 4 月の不安な時期にこれを頭に叩き込んだと思います。
個人的な話ですが 4 行 4 列よりサイズが大きくなると必ず計算ミスをします。どうにかしたい。

4 計画

今回はこれを Python で実装します。しかし、上記の説明のまま実装しようとする割と苦労します。例えば 5. で繰り返しの操作がありますが、コードで書くには行数を文字で置いたりして一般化する必要があります。そこで、以下のように書き換えます。

行列を縦 I 、横 J の大きさとする。

1. 左の列から右の列へ、 k 行目（はじめ $k=1$ ）から順に 0 でない数を探す。
はじめに見つけた数を n とする。
2. n がある行を k 行目と入れ替える。
3. k 行目を n で割る。
4. k 行目以外の行から k 行目を何倍かして引き、 n と同じ列の n 以外の数を 0 にする。
5. k に 1 を加え、1. からやり直す。 $k > I$ となるか、
 1. で 0 でない数が見つからなくなったとき、計算を終了する。

使いやすい文字をいくつか入れてみました。いっぺんに実装を考えるとときついですが、順番ごとに考えるとなんかいけそうな気がします。こっから Python を書いていきましょう！

1) 培風館 線形代数講義 [増補版] 木田雅成 著 P40 III.7 より引用

5 実装

5.1 Python での行列

5.2 リスト内包表記

Python で行列のような二次元リストを扱う際、リスト内包表記が便利です。リストの中にリストを入れる書き方です。例えば
$$\begin{cases} x + 2y - z = 2 \\ 2x - y + 3z = 9 \\ 3x + y + z = 8 \end{cases} \quad \text{を} \quad a = \begin{bmatrix} 1, 2, -1, 2, \\ 2, -1, 3, 9, \\ 3, 1, 1, 8 \end{bmatrix}$$
 と書きます。一行に直すと $a = [[1, 2, -1, 2], [2, -1, 3, 9], [3, 1, 1, 8]]$ となります。また、 i 行 j 列目、すなわち a_{ij} は今回はこの行列 a を演算対象とします。

5.3 numpy

他には `numpy` というライブラリを使う方法もあります。こちらでもリスト内包表記に似ていますが、こちらは行列自体を扱うことができます。更に簡約化はもちろん逆行列、行列式も 1 行で求めてくれます。よって今回の簡約化実装は車輪の最発明となります。

プログラムのソースコードをレポートに載せたいときは、`listings` パッケージを利用して `lstlisting` 環境で囲むと良いだろう。例えば、コード 1 のように書くと Python のコードがきれいに表示される。

コード 1: サンプルプログラム

```
1 def hello():
2     print("Hello, World!")
3
4 if __name__ == "__main__":
5     hello() # この行はコメントです
```

5.4 参考文献の明示

レポートを書く上では他の文献から情報を得ることも多いであろう。こういう時は、`thebibliography` 環境に参考文献として書籍やネットの記事を並べて出典を明確にするべきである。ネット上の情報の引用の際には `\url` マクロを利用して `https://www.example.com/` のように書くと良いだろう [1]。

参考文献を書くときは、`\bibitem` マクロを用いて文献を列挙する。このマクロに指定したラベルを `\cite` マクロで参照することで、文献番号を出力できる。

参考文献

- [1] 奥村晴彦, 黒木裕介, [改訂第 8 版] LaTeX2 ϵ 美文書作成入門, 技術評論社, 2020.

付録 A: 付録の書き方

本文に書くべきではない細かい計算や、導出は付録として別個にまとめることが多い。付録に入る際は`\appendix` マクロを用いる。その後、附番はアルファベットになり表番号や図番号の表記も変更される。なお、`\appendix` は 1 度書いた後はもう書く必要はない。